

TECHUP

Documento de arquitectura del backend

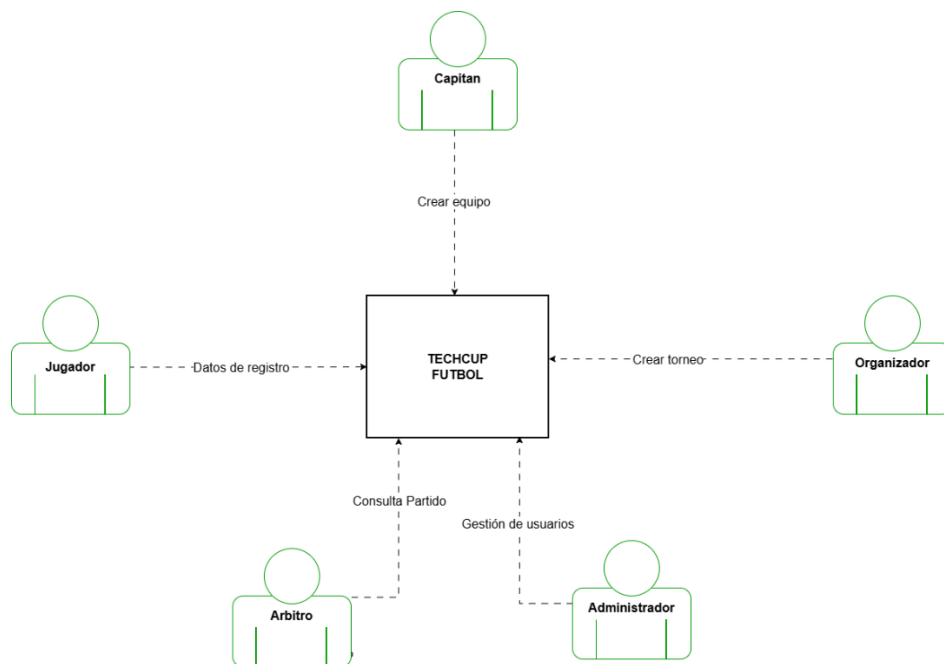
DOSW Company

1. Descripción del Servicio

1.1 Descripción general

TechCup es una plataforma de gestión de torneos de fútbol universitario que se desarrolló con Spring Boot. El sistema permite a organizadores crear y gestionar torneos, a capitanes registrar equipos y jugadores, a árbitros registrar resultados y eventos de partidos, y a jugadores consultar sus estadísticas. La aplicación tiene una API REST segura con autenticación JWT y OAuth2, respaldada por una base de datos PostgreSQL relacional.

1.2 Diagrama de contexto



Administrador: Gestiona los usuarios, asigna roles y supervisa el sistema.

Organizador: Crea los torneos, inicia y finaliza los torneos.

Capitán: Crea los equipos, puede invitar jugadores, registra su equipo en el torneo.

Jugador: Acepta invitaciones, consultar las estadísticas y gestionar su perfil deportivo.

Arbitro: Registra los resultados, eventos y alienaciones de los partidos

2. Tecnologías

2.1 Lenguaje y runtime

Java: Lenguaje que se utilizó para el código del backend

Sprint Boot: Para optimización en la configuración

Maven: Es la gestión de dependencias y ciclo de build

2.2 Frameworks y librerías

Spring Web MVC: Controladores REST, mapeo de las rutas HTTP

Spring Data JPA: Acceso a base de datos con Hibernate

Spring Security: Autenticación con JWT, OAuth2 y la a

JJWT: Genera firmas y validaciones de los tokens JWT

Spring OAuth2 Client: Para la autenticación con Google

MapStruct: Mapeo automático entre los modelos, DTOs y entidades JPA

Lombok: Eliminar código repetitivo como los getters, setters, builders y loggers

SpringDoc OpenAPI: Para generar documentación Swagger UI de forma automática

Hibernate Validator: Validación de campos en los DTOs

Spring Actuator: Métricas del sistema y verificar si el sistema está funcionando correctamente.

Jacoco: Cobertura del código con los test

SonarQube Scanner: Análisis estático de la calidad del código

2.3 Base de datos e infra

Base de datos: Para la gestión de base de datos se utilizó PostgreSQL 16

Contenedores: Para empaquetar, distribuir y ejecutar aplicaciones en contenedores se usó Docker.

Despliegue del Back: Para el despliegue se utilizó Azure

CI/CD: Para automatizar la construcción prueba y despliegue se usó GitHub Actions.

3. Funcionalidades

- Funcionales:

- RF01 – Gestión de Torneos El organizador puede crear un torneo con fecha inicial, fecha final, cantidad de equipos y costo por equipo. El torneo tendrá estados: Borrador, Activo, En progreso y Finalizado. El organizador puede iniciar, finalizar y consultar torneos.

- RF02 – Configuración del Torneo El organizador puede definir reglamento, fechas importantes, cierre de inscripciones, horarios de partidos, canchas disponibles y sanciones aplicables.

- RF03 – Registro de Usuarios Los estudiantes y graduados se registran con correo institucional; los familiares con correo Gmail. Cada usuario puede crear un perfil deportivo con posición de juego, número dorsal y foto.

- RF04 – Disponibilidad y búsqueda de jugadores Un jugador puede marcarse como disponible para ser contactado. Los capitanes pueden buscar jugadores por posición, semestre, edad, género, nombre e identificación.

- RF05 – Gestión de Equipos Un capitán puede crear un equipo, asignar nombre, subir escudo y definir colores del uniforme. Puede invitar jugadores y estos pueden aceptar o rechazar la invitación.

- RF06 – Validación de reglas de equipo El sistema valida que el equipo tenga entre 7 y 12 jugadores, ningún jugador pertenezca a dos equipos simultáneamente, más de la mitad de los miembros pertenezcan a los programas elegibles y no se permitan cambios de jugadores una vez iniciado el torneo.

- RF07 – Inscripción y pago El capitán sube el comprobante de pago (Nequi o efectivo). El organizador revisa el comprobante y cambia el estado del equipo: Pendiente a En revisión a Aprobado o Rechazado. Solo los equipos aprobados participan en el torneo.

- RF08 – Gestión de alineaciones El capitán puede seleccionar titulares y reservas, elegir formación táctica y ubicar jugadores visualmente en el campo antes de cada partido. Capitanes y jugadores pueden consultar la alineación del equipo rival.

- RF09 – Registro de resultados El organizador registra marcador, goleadores, tarjetas amarillas y tarjetas rojas de cada partido.

- RF10 – Consulta de partidos por árbitro El árbitro puede consultar la fecha, hora, cancha y equipos del partido que le corresponde arbitrar.

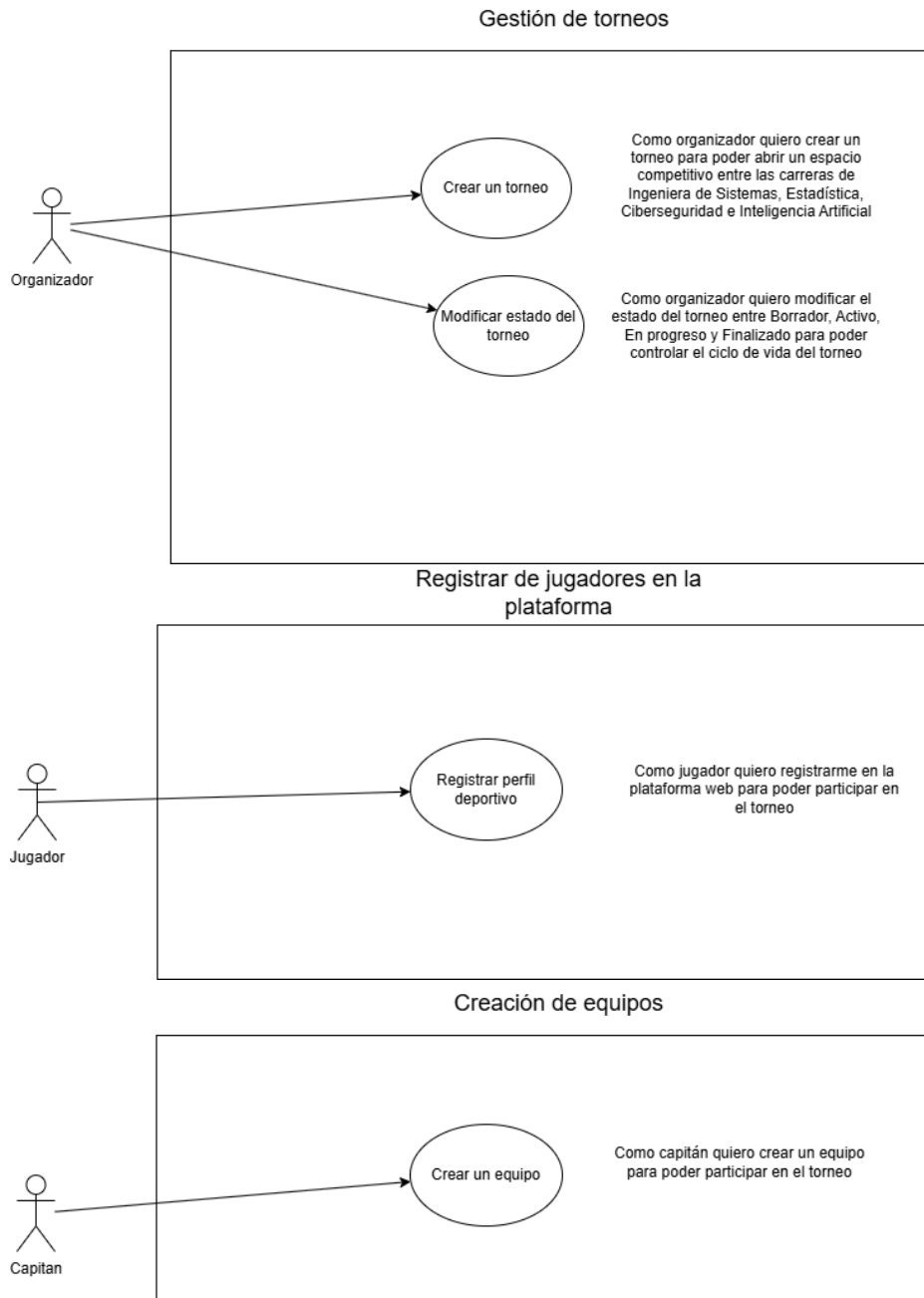
- RF11 – Tabla de posiciones El sistema calcula automáticamente por equipo: partidos jugados, ganados, empatados y perdidos, goles a favor, goles en contra, diferencia de gol y puntos totales.

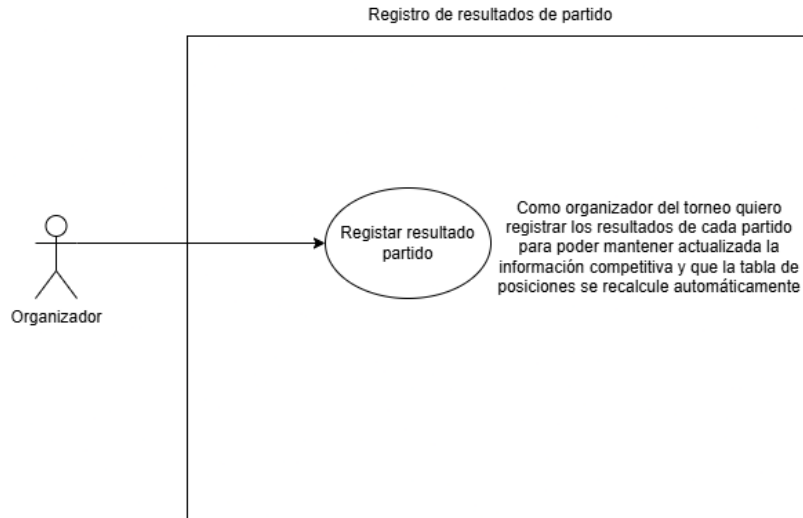
- RF12 – Llaves eliminatorias El sistema genera automáticamente los cruces iniciales de forma aleatoria y avanza las fases de cuartos de final, semifinal y final conforme se registran resultados.
- RF13 – Estadísticas del torneo El sistema permite consultar estadísticas generales del torneo activo, incluyendo el listado de máximos goleadores, cantidad de goles por jugador y ranking ordenado de mayor a menor. La información se actualiza automáticamente cada vez que se registran resultados de partidos.
- RF14 – Historial de torneos El sistema conserva la información de torneos anteriores y permite consultar por cada torneo: equipos participantes, tabla de posiciones final, resultados de partidos, campeones y estadísticas individuales. La información histórica no podrá ser modificada una vez el torneo esté finalizado.
- RF15 – Asignación de árbitros El organizador puede asignar un árbitro a cada partido programado del torneo, definiendo qué árbitro será responsable de dirigir el encuentro. El sistema debe evitar que un mismo árbitro sea asignado a dos partidos en el mismo horario.
- RF16 – Gestión de roles El administrador del sistema puede asignar, modificar o revocar roles de usuario (Capitán, Organizador, Árbitro), garantizando que cada usuario tenga los permisos correspondientes a su rol dentro del sistema. Un usuario podrá tener más de un rol si así lo define el administrador.
- RF17 – Consulta pública de información El sistema permite a usuarios no autenticados consultar información general del torneo, incluyendo reglamento, calendario de partidos, tabla de posiciones, resultados y llaves eliminatorias, garantizando transparencia en el desarrollo de la competencia.
- RF18 – Gestión de sanciones automáticas El sistema aplica automáticamente sanciones deportivas según el reglamento definido por el organizador. En caso de tarjeta roja, el jugador quedará suspendido para el siguiente partido del torneo, y el sistema impedirá su inclusión en la alineación correspondiente.
- RF19 – Notificación de invitaciones El sistema notifica a los jugadores cuando reciben una invitación para unirse a un equipo. La notificación debe indicar el nombre del equipo y del capitán que envía la invitación, permitiendo al jugador aceptarla o rechazarla.
- RF20 – Notificación de estado de inscripción El sistema notifica al capitán cuando el organizador cambie el estado del equipo (Pendiente, En revisión, Aprobado o Rechazado), indicando el resultado de la validación del comprobante de pago.
- RF21 – Notificación de asignación de partido El sistema notifica al árbitro cuando se le asigne un partido, indicando fecha, hora, cancha y equipos participantes, permitiendo su consulta dentro del módulo correspondiente.

- No funcionales:

- RNF01 – Seguridad y autenticación El sistema debe autenticar a los usuarios según su tipo: correo institucional para estudiantes, graduados y personal de la Escuela; correo Gmail para familiares. Debe implementarse control de roles y permisos por tipo de actor.
- RNF02 – Auditoría El sistema debe registrar un log de las acciones relevantes realizadas por los usuarios (creación de equipos, aprobación de pagos, registro de resultados, etc.) para trazabilidad.
- RNF03 – Arquitectura por capas El backend debe desarrollarse en Spring Boot con separación clara en capas: controladores, adaptadores, lógica de negocio y acceso a datos. La comunicación con el frontend se realizará mediante una API REST.
- RNF05 – Seguridad de almacenamiento de contraseñas Las contraseñas deben almacenarse utilizando un algoritmo de hashing seguro (por ejemplo, BCrypt), garantizando que no se almacenen en texto plano.
- RNF06 – Usabilidad La plataforma debe ser lo suficientemente intuitiva para que estudiantes sin experiencia técnica avanzada puedan inscribirse, unirse a equipos y consultar información del torneo sin asistencia externa.
- RNF07 – Disponibilidad La plataforma debe estar disponible durante todo el período del torneo, especialmente en fechas clave como cierre de inscripciones y registro de resultados.
- RNF08 – Escalabilidad básica El sistema debe soportar la carga de todos los equipos participantes en un torneo semestral sin degradación notable del rendimiento.
- RNF09 – Compatibilidad La aplicación debe ser compatible con las versiones actuales de los navegadores Google Chrome, Microsoft Edge y Mozilla Firefox.
- RNF10 – Respaldo de información La base de datos debe contar con respaldo automático diario para garantizar la recuperación de información ante fallos del sistema.
- RNF11 – Escalabilidad básica La arquitectura del sistema debe permitir aumentar la capacidad de usuarios concurrentes mediante escalamiento vertical del servidor sin necesidad de rediseño estructural.

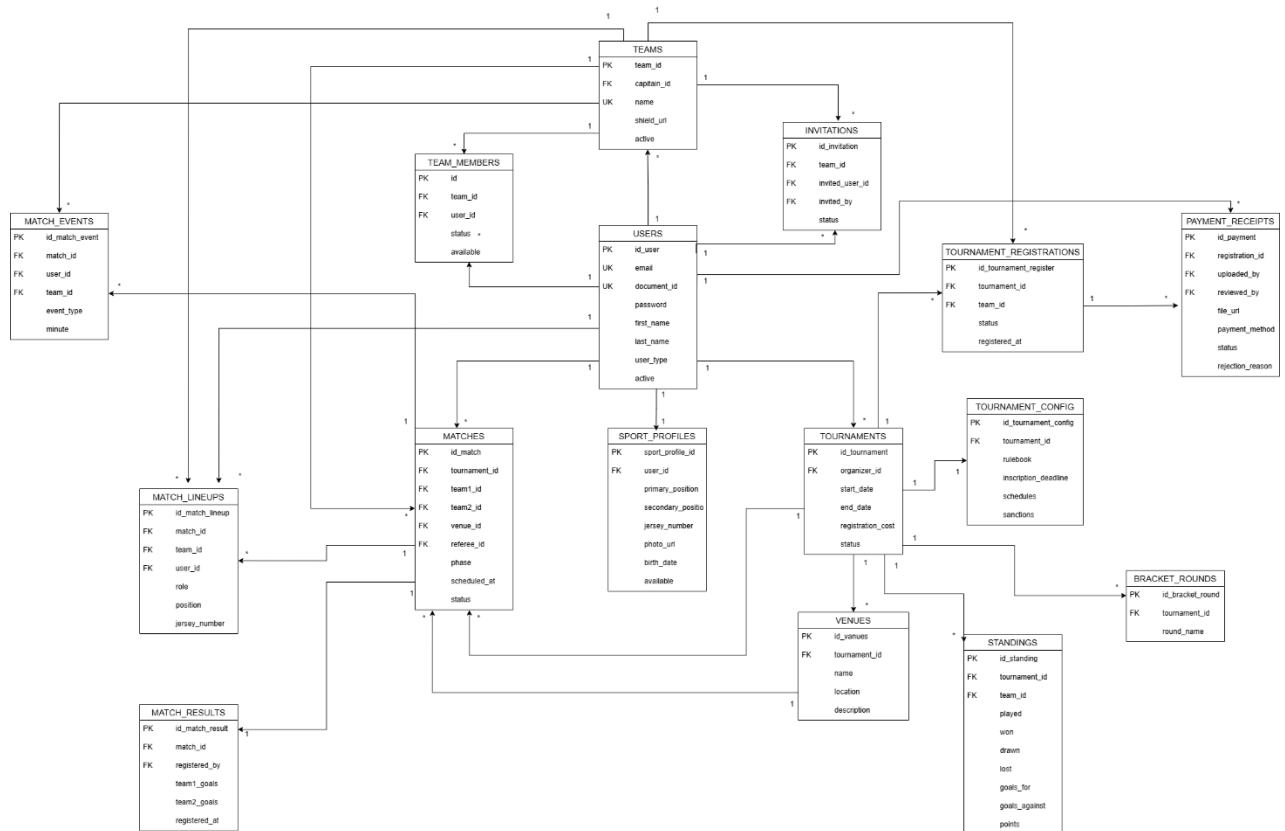
- Historias de usuario





4. Diagramas

4.1 Diagrama de base de datos



Teams: equipos de futbol, cada equipo tiene un capitán con su FK

Teams_memebers: La relación entre usuario y equipos con estado de membresía

Tournaments: Torneos con estado (SKETCH, ACTIVE, INPROGRESS, FINALIZED)

Tournaments_config: Configuración adicional que se le quiera poner al torneo.

Tournaments_registrations: La inscripción de un equipo en un torneo.

Venues: Canchas o lugares que estén asociados a un torneo

Matches: Los partidos programados para los equipos (GROUPSTAGE, QUARTERFINALS, SEMIFINALS, FINAL)

Match_results: Los goles por equipo

Match_events: Los eventos que pasen en el partido (GOL, AMARILLA, ROJA)

Lineups: Alineaciones de un equipo en el partido como su titular y suplente

Standings: La tabla de posiciones con puntos, goles partidos jugados

Payments: El comprobante de pago para la inscripción

Sport_profiles: El perfil deportivo de un jugador con su posición, disponibilidad y semestre

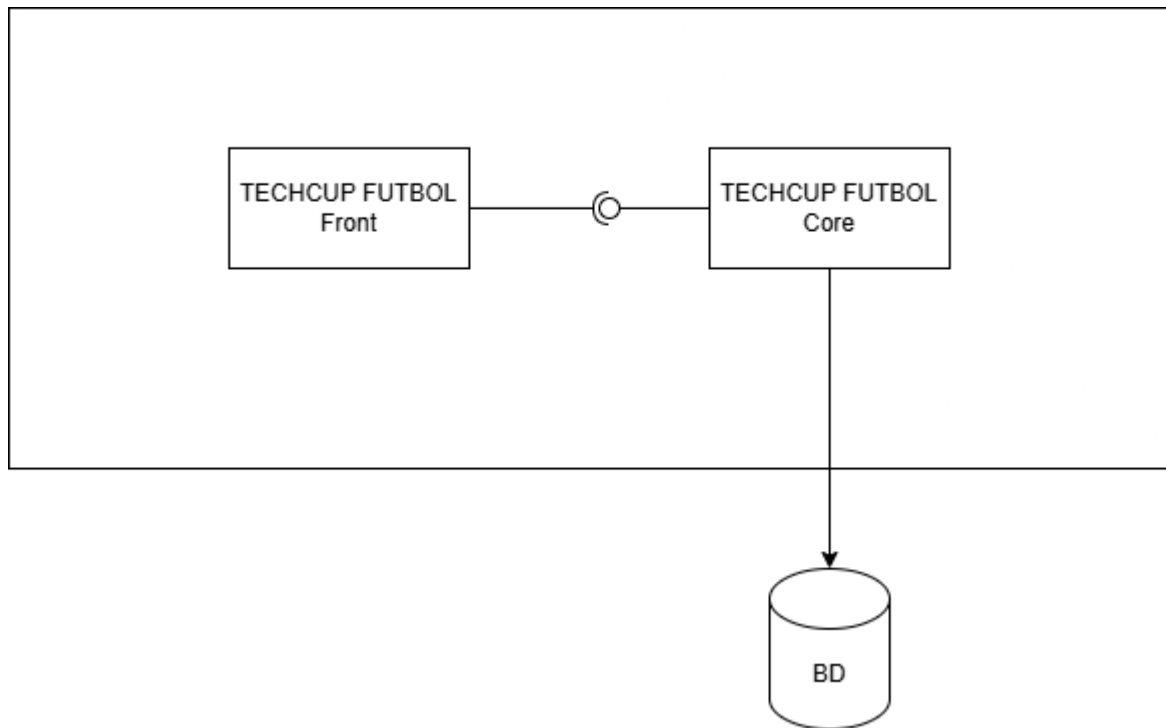
Invitations: Las invitaciones enviadas por los capitanes

Bracket_rounds: Las rondas que se generan para el bracket del torneo

4.2 Diagrama de clases

El modelo de dominio sigue una arquitectura en capas donde los modelos del core (paquete core.model) son independientes de la persistencia. También se usaron los patrones de diseño Builder y Strategy.

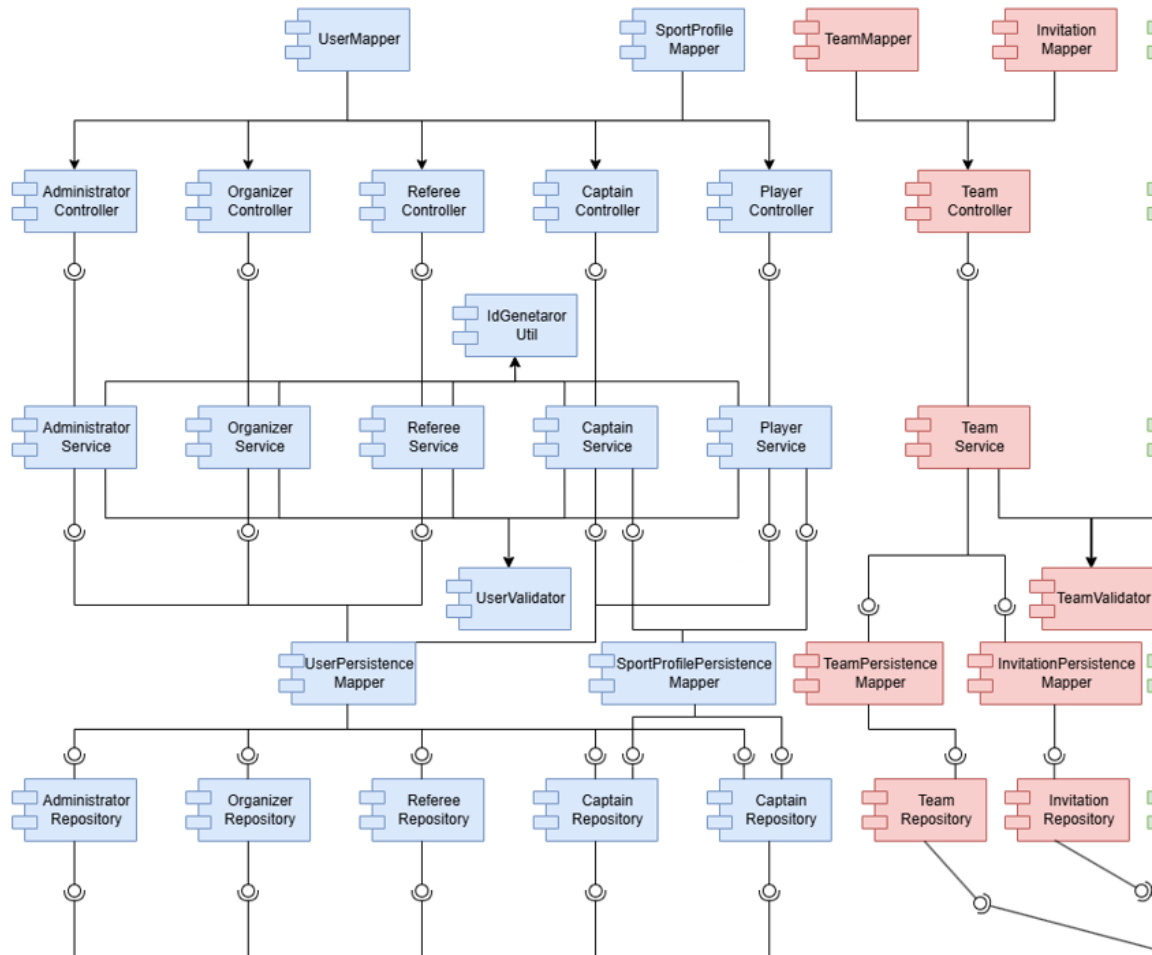
El sistema esta dividido en 3 capas principales FrontEnd, BackEnd y la Data Base

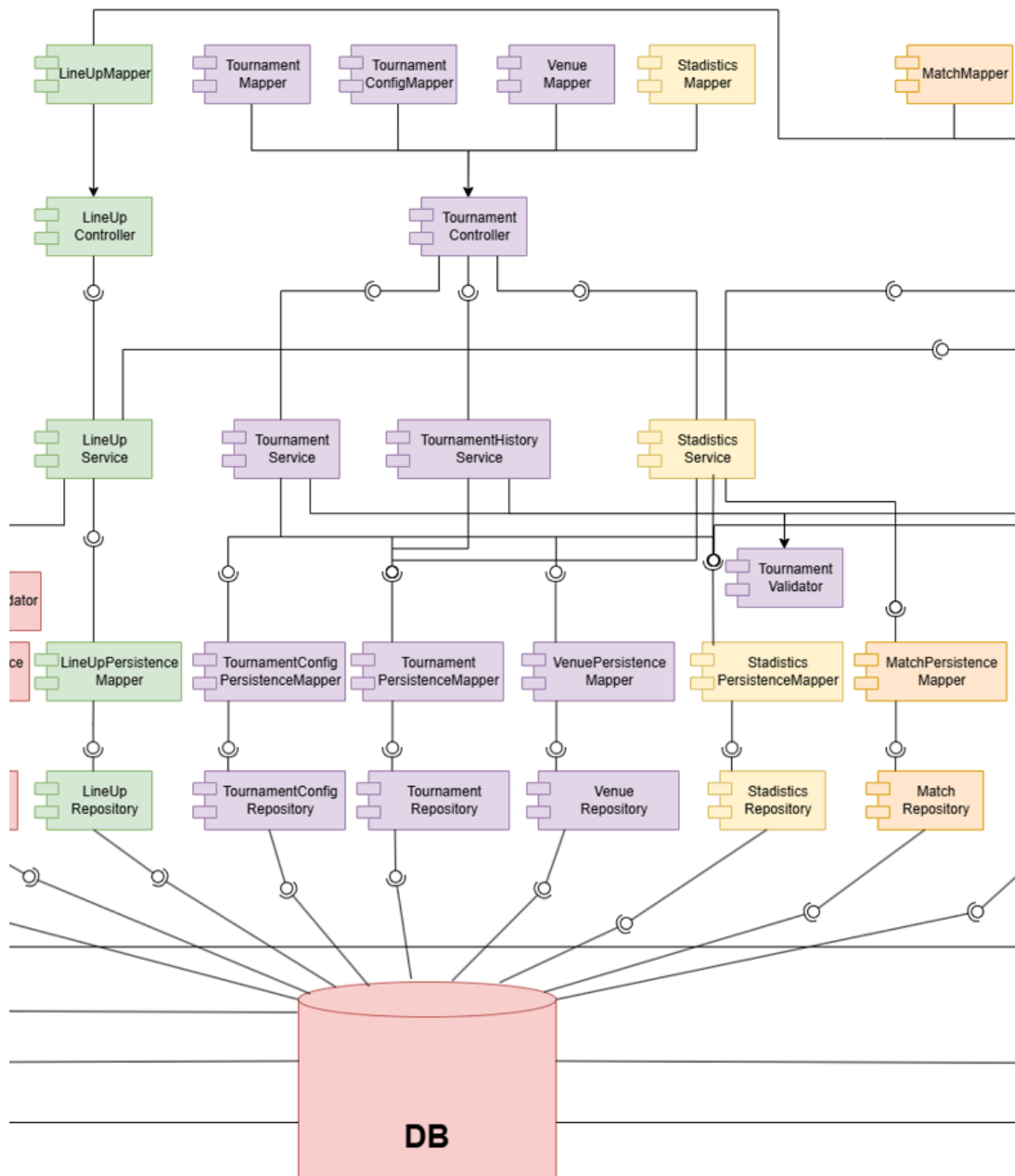


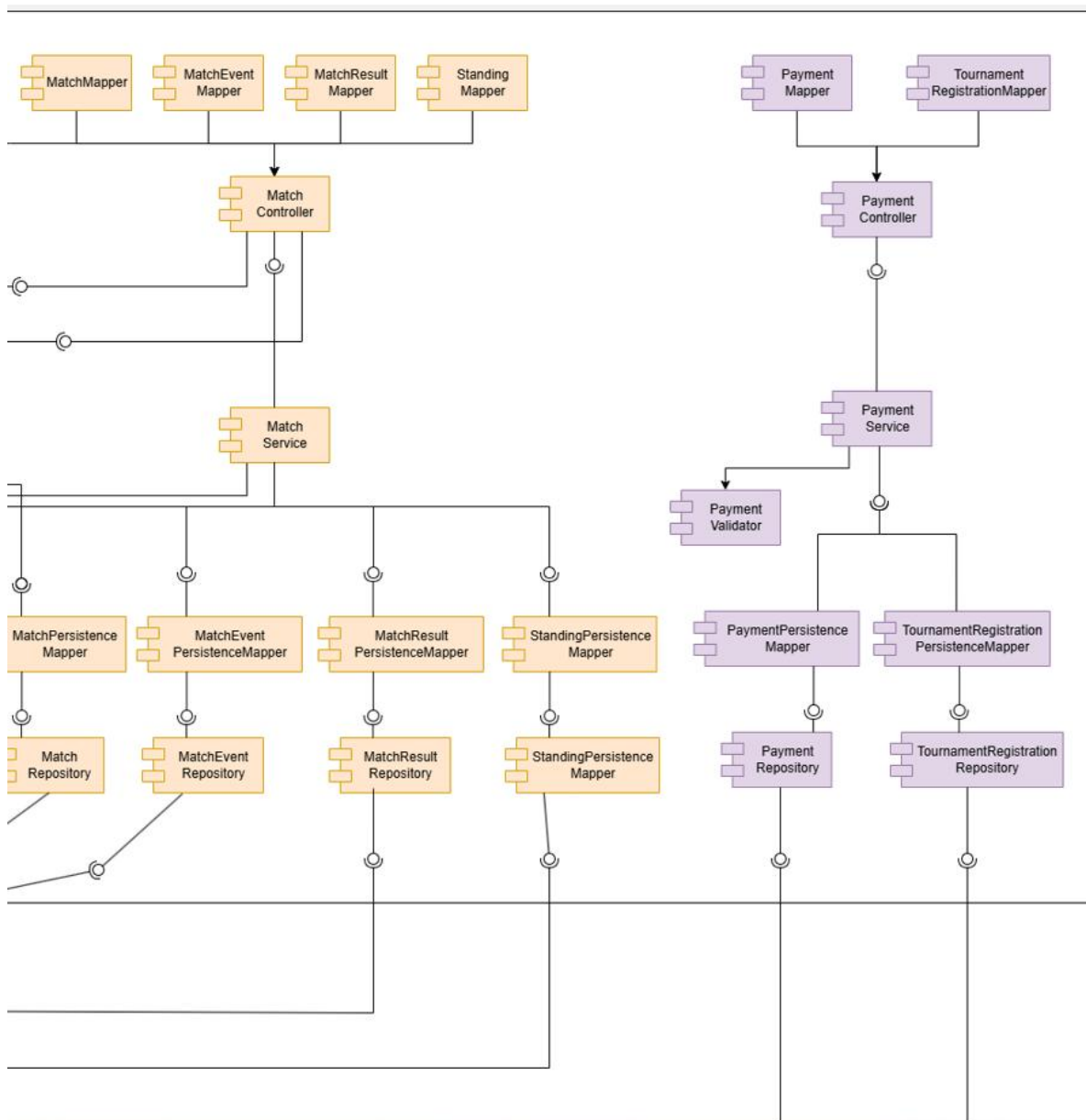
4.4 Diagrama de componentes específicos

El sistema está dividido en 4 capas principales que se comunican de forma unidireccional Controller Service Repository/Persistence Database

TECHCUP

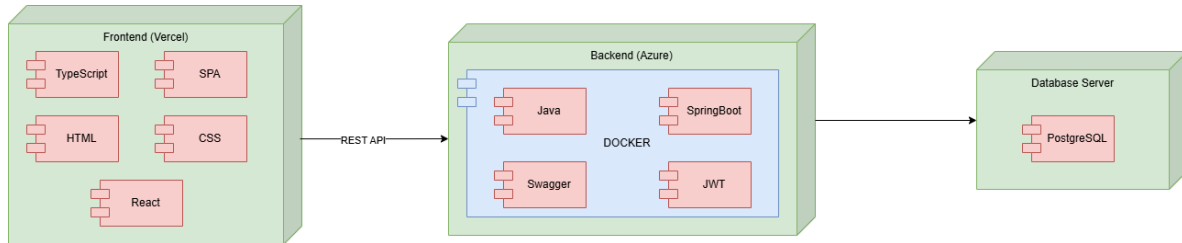






4.5 Diagrama de despliegue

El diagrama representa una arquitectura distribuida basada en el modelo cliente-servidor, desplegada en la nube y organizada en tres capas principales: Frontend, Backend y Base de Datos, comunicadas a través de una API REST.



5. Funcionalidades expuestas

5.1 Autenticación

Método	Endpoint	Roles	Request Body	Response
POST	/auth/register	Público	RegisterRequestDTO	201 LoginResponseDTO
POST	/auth/login	Público	LoginRequestDTO	200 LoginResponseDTO
GET	/auth/oauth2/google	Público	—	Redirect Google OAuth2

5.2 Torneos

Método	Endpoint	Roles requeridos	Response
POST	/api/tournaments	ORGANIZER, ADMINISTRATOR	201 TournamentResponseDTO

Método	Endpoint	Roles requeridos	Response
GET	/api/tournaments	Todos los roles	200 List<TournamentResponseDTO>
GET	/api/tournaments/{id}	Todos los roles	200 TournamentResponseDTO
GET	/api/tournaments/finalized	Todos los roles	200 List<TournamentResponseDTO>
PATCH	/api/tournaments/{id}/start	ORGANIZER, ADMINISTRATOR	200 TournamentResponseDTO
PATCH	/api/tournaments/{id}/finish	ORGANIZER, ADMINISTRATOR	200 TournamentResponseDTO
POST	/api/tournaments/{id}/config	ORGANIZER, ADMINISTRATOR	200 TournamentConfigResponseDTO
GET	/api/tournaments/{id}/config	Todos los roles	200 TournamentConfigResponseDTO
POST	/api/tournaments/{id}/venues	ORGANIZER, ADMINISTRATOR	201 VenueResponseDTO
GET	/api/tournaments/{id}/venues	Todos los roles	200 List<VenueResponseDTO>
GET	/api/tournaments/{id}/statistics	Todos los roles	200 TournamentStatisticsResponseDTO
GET	/api/tournaments/historial	Todos los roles	200 List<TournamentResponseDTO>
GET	/api/tournaments/{id}/historial	Todos los roles	200 TournamentHistoryResponseDTO

5.3 Partidos

Método	Endpoint	Roles requeridos	Response
POST	/api/matches	ORGANIZER, ADMINISTRATOR	201 MatchResponseDTO
POST	/api/matches/result	ORGANIZER, ADMINISTRATOR, REFEREE	200 MatchSummaryResponseDTO
GET	/api/matches/{id}/summary	Todos los roles	200 MatchSummaryResponseDTO
POST	/api/matches/event	REFEREE	201 MatchEventResponseDTO
POST	/api/matches/{id}/lineup	CAPTAIN	200 List<LineUpResponseDTO>
GET	/api/matches/{id}/lineup	Todos los roles	200 List<LineUpResponseDTO>
GET	/api/matches/referee/{id}	Todos los roles	200 List<MatchResponseDTO>
POST	/api/matches/bracket/{id}	ORGANIZER, ADMINISTRATOR	201 List<MatchResponseDTO>
GET	/api/matches/tournament/{id}	Todos los roles	200 List<MatchResponseDTO>
GET	/api/matches/tournament/{id}/history	Todos los roles	200 List<MatchResponseDTO>
GET	/api/matches/tournament/{id}/standings	Todos los roles	200 List<StandingResponseDTO>
GET	/api/matches/tournament/{id}/top-scorers	Todos los roles	200 List<TopScorerResponseDTO>

5.4 Equipos

Método	Endpoint	Roles requeridos	Response
POST	/api/teams	CAPTAIN	201 TeamResponseDTO
GET	/api/teams/{id}	Todos los roles	200 TeamResponseDTO
GET	/api/teams/tournament/{id}	Todos los roles	200 List<TeamResponseDTO>
POST	/api/teams/{id}/invite	CAPTAIN	201 InvitationResponseDTO
PATCH	/api/teams/invitations/{id}/respond	PLAYER	200 InvitationResponseDTO
GET	/api/teams/invitations/player/{id}	PLAYER, CAPTAIN	200 List<InvitationResponseDTO>
GET	/api/teams/players/available	CAPTAIN	200 List<SportProfileResponseDTO>
POST	/api/teams/{id}/validate	CAPTAIN, ORGANIZER	200 (vacío)

6. Manejo de errores

Excepción	HTTP Status	Código	Escenario
TournamentNotFoundException	404 Not Found	404	Torneo no existe en la base de datos
VenueNotFoundException	404 Not Found	404	Cancha/sede no encontrada
UserNotFoundException	404 Not Found	404	Usuario no encontrado por ID
TeamNotFoundException	404 Not Found	404	Equipo no encontrado por ID
TournamentValidationException	400 Bad Request	400	Fechas inválidas, transición de estado incorrecta
TournamentFinalizedException	400 Bad Request	400	Se intenta modificar un torneo ya finalizado
TournamentNotFinalizedException	400 Bad Request	400	Se pide historial de torneo no finalizado

Excepción	HTTP Status	Código	Escenario
UserValidationException	400 Bad Request	400	Datos del usuario inválidos (email, contraseña)
IllegalArgumentException	400 Bad Request	400	Argumento inválido en cualquier operación
MethodArgumentNotValidException	400 Bad Request	400	Campos @Valid fallidos en el DTO
MatchResultAlreadyExistsException	409 Conflict	409	Se intenta registrar resultado de un partido que ya tiene uno
InvalidAdminException	403 Forbidden	403	Usuario sin rol ADMINISTRATOR intenta operación de admin
UnauthorizedRoleAssignmentException	403 Forbidden	403	Se intenta asignar el rol ADMINISTRATOR (no permitido)
BadCredentialsException	401 Unauthorized	401	Email o contraseña incorrectos en /auth/login
UsernameNotFoundException	401 Unauthorized	401	Email no registrado — mismo mensaje que BadCredentials
DisabledException / LockedException	403 Forbidden	403	Cuenta deshabilitada o bloqueada